

Confidence-Constrained Engineering Economics

A Bayesian Decision Framework for Architecture, Deployment, Security, Sovereignty, and AI Inference

Guillaume Harbonnier

Systems Engineering, Quality Governance, and Applied Research Program

Foundation article draft - May 2026

Abstract

Modern software-intensive organizations face a decision problem that existing frameworks do not adequately model. Engineers do not only choose components and deployment platforms; they arbitrate among SaaS, on-premise, hybrid cloud, sovereign cloud, security exposure, staging representativeness, deployment patterns, AI-token markets, internal inference capacity, delivery margin, and residual operational risk. These variables are coupled and non-linear. A lower infrastructure bill can increase security, quality, or sovereignty risk. A highly representative pre-production environment can reduce residual risk but destroy bid margin. A SaaS option can accelerate delivery but introduce supplier, jurisdictional, data, and exit-strategy dependencies. An external AI API can reduce time-to-market but create strategic dependence on token pricing, provider policy, data exposure, and inference availability.

This article proposes a foundation model for **confidence-constrained engineering economics**: a decision framework that integrates cost, risk, evidence, security, sovereignty, staging economics, deployment strategy, and AI compute under uncertainty. The core decision is not to minimize cost, but to identify the minimum cost that buys a defensible level of confidence. We formalize this through a Bayesian posterior gate where release, bid, architecture, and AI-inference decisions are accepted only if posterior probabilities satisfy joint constraints on residual risk, adjusted margin, team blocking, security exposure, and sovereignty exposure.

The contribution is a conceptual and mathematical foundation: FinOps measures what technology costs; this framework formalizes what that cost buys in terms of defensible confidence.

Keywords: engineering economics; systems engineering; software economics; residual risk; Bayesian decision-making; SaaS; on-premise; sovereign cloud; DevSecOps; staging; pre-production; blue/green; canary; AI inference; token economics; cloud repatriation; Master Test Plan; confidence threshold.

Contents

1. Introduction
2. Research Motivation and Problem Statement
3. Related Work
4. Core Thesis: Confidence-Constrained Engineering Economics

5. Formal Model
 6. Environment Economics: Staging as Risk Compression
 7. Deployment Patterns as Risk-Release Controls
 8. SaaS versus On-Premise as a Governance Decision
 9. AI Inference as Strategic Infrastructure
 10. Stakeholder Governance Model
 11. Decision Artifacts for a Master Test Plan
 12. Illustrative Scenario
 13. Research Agenda
 14. Limitations
 15. Conclusion
-

1. Introduction

The modern engineering organization increasingly operates under a set of constraints that were historically treated separately: architecture, cost, quality, security, compliance, sovereignty, delivery speed, customer commitments, and operational risk. The rise of cloud computing exposed one dimension of this problem: technology spend is no longer a mostly fixed capital expenditure but a variable operational flow. FinOps emerged to provide visibility, accountability, and optimization over that flow. However, treating FinOps as a cost-reduction practice is too narrow for critical systems engineering.

The central claim of this article is that engineering economics needs a new governing question. The relevant question is no longer only: *What is the cheapest architecture?* The relevant question is: *What architecture, environment, deployment pattern, security posture, and AI-inference strategy buys the required level of confidence without destroying delivery margin or strategic autonomy?*

This shift matters because the economic model of software delivery is now inseparable from uncertainty. A low-cost SaaS choice may be economically attractive but may expose critical data to supplier, jurisdictional, or contractual risks. An on-premise solution may appear sovereign but can be less secure in practice if the organization lacks patching, monitoring, incident response, and security expertise. A production-like staging environment can reduce residual risk, but its cost grows super-linearly when data, interfaces, observability, access controls, secrets, and parallel versions must all be replicated. An AI API can compress development time, but the organization becomes exposed to token pricing, policy changes, rate limits, model drift, and strategic dependence on a narrow compute supply chain.

In parallel, the Master Test Plan (MTP) and test strategy must evolve. The MTP should not be a static list of test activities. It should encode an economically defensible threshold configuration under which a decision becomes acceptable. In engineering economics terms, testing is threshold calibration under uncertainty — the MTP is not a list of activities but a confidence purchase plan.

2. Research Motivation and Problem Statement

2.1 The Missing Model

Existing disciplines provide partial answers. FinOps explains how to measure and govern technology spend. DevSecOps and the NIST Secure Software Development Framework (SSDF) explain how to integrate security into software development life cycles. Cloud sovereignty frameworks explain jurisdictional and strategic control constraints. Deployment patterns such as blue/green, rolling updates, canary, feature flags, and shadow traffic reduce release risk. Bayesian software testing literature models uncertainty in test selection and regression testing. AI economics literature models token pricing, inference cost, latency, and compute governance.

Engineering economics as a discipline — built on Boehm’s cost estimation research, McConnell’s software estimation practice, and Hubbard’s applied measurement frameworks — provides cost modeling tools. What is missing is an integrated model that connects all of these in the actual decision space of engineering leadership under uncertainty.

The gap can be stated precisely:

Organizations lack an operationally usable decision framework for selecting SaaS, on-premise, hybrid architectures, staging depth, deployment patterns, security controls, and AI inference strategies under joint constraints of cost, margin, residual risk, sovereignty, security, and delivery flow.

2.2 Research Question

This foundation article addresses the following research question:

How can software-intensive organizations make economically rational and evidence-based engineering decisions under uncertainty when cost, security, sovereignty, staging representativeness, deployment strategy, AI inference dependency, and residual operational risk interact non-linearly?

2.3 Contributions

The article makes four foundation-level contributions:

1. It reframes engineering economics from cost optimization to confidence-constrained governance.
2. It proposes a Bayesian residual-risk and margin gate for architecture, release, bid, staging, and AI-inference decisions, with explicit treatment of prior elicitation and dependency structure.
3. It introduces environment economics as a first-class dimension: staging and pre-production environments are not servers, but risk-compression and team-flow assets.
4. It maps AI token markets and sovereign compute into the same trade-off space as SaaS, on-premise, security, and cloud sovereignty.

3. Related Work

3.1 FinOps and Automated Cost Governance

The FinOps Foundation defines FinOps as an operational and cultural practice that enables timely data-driven decisions and creates financial accountability through collaboration between engineering, finance, technology, and business teams. Its principles explicitly recognize trade-offs among cost, quality, and speed. Its lifecycle is structured around the iterative phases Inform, Optimize, and Operate.

ABACUS extends this toward automation: Automated Budget Analysis and Cloud Usage Surveillance sets budgets, enforces them by blocking deployments when thresholds are

breached, and leverages Infrastructure-as-Code to expose expected costs before resources are deployed. This is relevant to engineering governance because it turns cost from a retrospective finance report into a pipeline-time control signal.

However, ABACUS is still primarily cost-threshold oriented. In critical systems, a cost threshold cannot be interpreted independently from risk. Blocking a deployment because it breaches a budget may be correct for non-critical waste, but wrong when the additional cost buys a large reduction in residual operational, security, or contractual risk. The missing layer is risk-adjusted cost governance.

3.2 Engineering Economics

The engineering economics tradition provides the appropriate academic home for this work. Boehm's COCOMO and its successors formalized cost estimation under uncertainty for software-intensive systems. McConnell's software estimation practice extended this to schedule and scope uncertainty. Hubbard's applied measurement approach — measuring what matters, including risk — provides the epistemological grounding: any claim that something is "immeasurable" typically reflects a failure of measurement design, not a property of the quantity itself.

These traditions are largely cost-focused. The present article extends them by making *confidence* the primary quantity being purchased, with cost as the constraint.

3.3 SaaS, On-Premise, Cloud Security, and Sovereignty

Cloud adoption introduces both security benefits and risks. ENISA notes that cloud concentration can create attractive targets, while cloud-based defenses can also be more robust, scalable, and cost-effective. This undermines simplistic claims such as "on-premise is always safer" or "cloud is always more secure." Security is a capability, not a location.

Sovereignty introduces another dimension. The European Commission's Cloud Sovereignty Framework defines sovereignty objectives for cloud services and draws on initiatives such as Gaia-X, ENISA, NIS2, DORA, and national trusted cloud strategies. For critical data, public-sector systems, citizen identity, biometrics, health, justice, finance, and election-related systems, data locality, jurisdictional exposure, operational control, auditability, and exit capability become first-order decision variables.

Cloud repatriation literature adds a counterpoint to cloud-first narratives. Some workloads move back from public cloud to private cloud, colocation, or on-premise environments because of cost predictability, performance, security, control, or regulatory pressure. Placement decisions must be periodically re-evaluated as usage, cost, risk, regulation, and organizational capability evolve.

3.4 Security Governance

The NIST SSDF recommends high-level secure software development practices integrable into each SDLC implementation. It matters not only for developers but also for organizations that procure software, providing a common language for producers, acquirers, suppliers, and managers. Security champions and DevSecOps research point in the same direction: security must be distributed across agile teams, with roles, training, governance, tooling, and cultural mechanisms that allow security to be addressed at development tempo.

In the present framework, security is not an after-the-fact control. It is an influence factor that changes architecture, staging, test strategy, bid pricing, deployment pattern, and evidence requirements.

3.5 Deployment Patterns as Residual-Risk Controls

Deployment patterns are often described as DevOps techniques, but in a systems engineering view they are residual-risk controls. Blue/green deployment shifts traffic between two

equivalent environments and provides near zero-downtime and rollback capability. Kubernetes rolling updates use parameters such as `maxUnavailable` and `maxSurge` to control availability during replacement. Feature flags decouple deployment from exposure, enabling gradual activation and rapid logical rollback. Canary, ring deployments, A/B testing, shadow traffic, and parallel run each buy a different kind of confidence at a different cost.

3.6 Bayesian Testing and Risk-Based Selection

Bayesian frameworks for regression testing model the probability that test cases reveal faults and use that probability for prioritization or selection. Risk-based test selection models combine defect probabilities and defect costs to estimate potential risk decrement. Bayesian Belief Networks model software quality and project risk.

The present work generalizes these: architecture choice, staging depth, security control level, deployment pattern, AI inference placement, bid pricing, and release acceptance are all threshold decisions under uncertainty, not only test execution decisions.

3.7 AI Tokens, Inference Economics, and Compute Governance

AI adoption can be implemented via external APIs, managed platforms, open-weight models, internal models, or public/non-profit compute infrastructures. Each option trades off token cost, latency, quality, data exposure, vendor dependence, sovereignty, compute utilization, and organizational capability.

Recent work on inference economics formalizes the trade-off between cost per token and serial token generation speed, accounting for arithmetic, memory bandwidth, network bandwidth, latency, parallelism, and batch size. Work on LLM pricing models token budgets and product menus under heterogeneous task valuations. OECD market analysis provides empirical indicators based on model characteristics, prices, providers, and downstream applications. Compute governance literature argues that AI-relevant compute is detectable, excludable, quantifiable, and concentrated in a limited supply chain, making it a governance lever but also a centralization risk. EuroHPC AI Factories illustrate a public-infrastructure response: shared supercomputing capacity supporting startups, SMEs, industry, and research communities.

For engineering governance, the implication is direct: AI inference is not merely a software dependency. It is a strategic infrastructure dependency.

4. Core Thesis: Confidence-Constrained Engineering Economics

4.1 From Cost Optimization to Confidence Purchase

Classical cost optimization asks whether a resource is idle, overprovisioned, or too expensive. Confidence-constrained engineering economics asks a different question:

What level of cost is justified because it buys a measurable reduction in residual risk, improves reversibility, protects margin, preserves team flow, improves security posture, or reduces strategic dependence?

This moves the discipline from a one-dimensional cost question to a multi-objective governance system. The key distinction is shown in Table 1.

Table 1: From cost-centric to confidence-constrained engineering economics.

Dimension	Cost-Centric	Confidence-Constrained
Primary question	How do we reduce spend?	What confidence does spend buy?

Dimension	Cost-Centric	Confidence-Constrained
Control signal	Budget, utilization, forecast	Budget, posterior risk, margin, evidence
Decision scope	Cloud resources	Architecture, staging, security, release, AI inference
Failure mode	Overspending	False economy: cheap design with high residual risk
Optimality	Minimum cost	Minimum cost under confidence constraints
Stakeholders	Finance, engineering, operations	Finance, engineering, security, QA/BA, product, sales, legal, executive

4.2 Engineering Trade-Off Map

The key insight is that SaaS/on-premise, security, staging, deployment, and AI inference are not independent choices. They are coupled decision levers feeding a single governance question: does the proposed strategy buy sufficient confidence at acceptable cost and margin?

Each lever interacts with the others. SaaS choice affects data sovereignty, which affects staging depth requirements, which affects environment cost, which affects margin. AI inference mode affects data exposure risk, which affects security controls required, which affects bid price competitiveness. These couplings are the central modeling challenge.

5. Formal Model

5.1 Decision Variables

Let a candidate engineering strategy be represented by a vector

$$\mathbf{a} = (a_{\text{arch}}, a_{\text{env}}, a_{\text{sec}}, a_{\text{rel}}, a_{\text{ai}}, a_{\text{obs}}, a_{\text{data}}),$$

where a_{arch} denotes placement choice (SaaS, cloud, sovereign cloud, hybrid, on-premise), a_{env} denotes environment strategy (local, preview, integration, blue/green qualification, production-like pre-prod), a_{sec} denotes security control level, a_{rel} denotes release pattern, a_{ai} denotes AI inference mode, a_{obs} denotes observability level, and a_{data} denotes test data and data-governance strategy.

Each strategy produces distributions rather than deterministic scalars:

$$C(\mathbf{a}), R(\mathbf{a}), V(\mathbf{a}), M(\mathbf{a}), S(\mathbf{a}), G(\mathbf{a}), B(\mathbf{a}),$$

where C is total cost, R residual operational risk, V value, M adjusted margin, S security exposure, G sovereignty exposure, and B team-blocking exposure.

5.2 Adjusted Margin

A naive bid margin is:

$$M_{\text{naive}} = \text{Price} - \text{DeliveryCost} - \text{InfrastructureCost}.$$

The proposed adjusted margin includes testing, security, environment, AI inference, expected residual risk, and a black swan buffer:

$$\begin{aligned} M_{\text{adj}}(\mathbf{a}) = & \text{Price} - C_{\text{delivery}}(\mathbf{a}) - C_{\text{infra}}(\mathbf{a}) - C_{\text{test}}(\mathbf{a}) \\ & - C_{\text{security}}(\mathbf{a}) - C_{\text{env}}(\mathbf{a}) - C_{\text{ai}}(\mathbf{a}) \\ & - \mathbb{E}[\text{Loss}_{\text{residual}}(\mathbf{a})] - B_{\text{blackswan}}. \end{aligned}$$

The black swan buffer $B_{\text{blackswan}}$ accounts for low-probability, high-impact events (supply chain attacks, critical zero-days, sovereign regulatory changes, AI provider policy shifts) that resist estimation from historical data. It is calibrated as a percentage of project value, informed by sector-specific incident benchmarks, and reviewed at each programme gate. A project can be profitable under naive margin and unattractive under risk-adjusted margin. The black swan buffer makes this gap explicit rather than hidden.

5.3 Prior Elicitation

The posterior belief requires a prior $p(\theta \mid \mathbf{a})$ over latent risk parameters θ — including defect escape probability, security exposure, interface instability, operational failure impact, data leakage exposure, and deployment reversibility.

These priors cannot be assumed; they must be elicited. Two operationally viable methods are:

Structured expert elicitation. The Cooke classical model and the SHELF protocol provide calibrated approaches to aggregating expert probability estimates. For each risk dimension, subject matter experts (security, QA, platform, operations) provide estimates that are then weighted by calibration scores against known reference questions.

Reference class forecasting. Historical data from similar programmes — defect escape rates, incident frequency, staging-to-production divergence — can inform empirical priors. This is the Hubbard approach: even sparse data constrains priors meaningfully.

Both methods should be applied in combination. The key discipline is that priors are stated explicitly and are revisable, rather than hidden inside intuitive point estimates.

5.4 Bayesian Update

Let E denote accumulated evidence: requirements traceability, tests, logs, traces, incidents, static analysis, threat modeling, performance data, staging runs, canary telemetry, and customer acceptance results. The posterior belief is:

$$p(\theta \mid E, \mathbf{a}) = \frac{p(E \mid \theta, \mathbf{a}) p(\theta \mid \mathbf{a})}{p(E \mid \mathbf{a})}.$$

5.5 Decision Gate

The decision gate is expressed as joint posterior constraints:

$$\begin{aligned} \Pr(R(\mathbf{a}) \leq \tau_R \mid E) &\geq \gamma_R, \\ \Pr(M_{\text{adj}}(\mathbf{a}) \geq \tau_M \mid E) &\geq \gamma_M, \\ \Pr(S(\mathbf{a}) \leq \tau_S \mid E) &\geq \gamma_S, \\ \Pr(G(\mathbf{a}) \leq \tau_G \mid E) &\geq \gamma_G, \\ \Pr(B(\mathbf{a}) \leq \tau_B \mid E) &\geq \gamma_B. \end{aligned}$$

Typical confidence thresholds γ are set at 0.90 or 0.95 depending on criticality and regulatory exposure. The key is not the specific value but the principle: release and bid decisions require posterior evidence, not point-estimate optimism.

Dependency structure. The joint constraint requires careful treatment. Individual dimensions are not independent: SaaS choice co-determines sovereignty exposure and security posture; staging depth co-determines both residual risk and team-blocking exposure. Where dimensions are correlated, the joint probability is lower than the product of marginals. As a tractable first approximation, a Gaussian copula can model pairwise correlations between risk dimensions, with correlation parameters estimated from historical programme data or expert elicitation. Future calibration work should test whether a vine copula structure better captures the dependency topology.

Regulatory hard constraints. Certain regulatory requirements (GDPR data residency, NIS2 incident reporting mandates, EU AI Act transparency obligations) operate as hard constraints, not probabilistic thresholds. These should be encoded as feasibility filters that eliminate non-compliant strategies before the optimization is evaluated. The posterior gate applies only to the feasible strategy set.

5.6 Optimization

The rational strategy is the minimum-cost strategy that satisfies posterior constraints:

$$\mathbf{a}^* = \arg \min_{\mathbf{a} \in A} \mathbb{E}[C(\mathbf{a})]$$

subject to:

$$\Pr(R \leq \tau_R, M_{\text{adj}} \geq \tau_M, S \leq \tau_S, G \leq \tau_G, B \leq \tau_B \mid E, \mathbf{a}) \geq \Gamma,$$

and all regulatory hard constraints satisfied.

Practical approximation. The full optimization is computationally intractable for large strategy spaces. Three practical approaches are available in combination: (i) Monte Carlo simulation over sampled strategy vectors to estimate joint constraint satisfaction; (ii) sensitivity analysis to identify which decision variables most influence the posterior — typically a small subset dominates; (iii) greedy staged search that first selects the dominant dimension (usually placement or security posture) and conditions subsequent choices on that selection. The goal is not a mathematically optimal solution but a demonstrably defensible one.

6. Environment Economics: Staging as Risk Compression

6.1 Why Environment Cost Grows Super-Linearly

A staging or pre-production environment is often mistakenly treated as a server cost. In serious engineering programmes, the real cost is the cost of representative execution capacity. To make a staging environment useful, an organization may need representative data, realistic interfaces, partner simulators, versioned APIs, secrets, access rights, observability, audit logs, data masking, performance capacity, rollback capability, and governance.

Let n be the number of parallel environments, r the representativeness level, and d the number of external or critical dependencies. A simplified environment cost function is:

$$C_{\text{env}}(n, r, d) = C_0 + n(C_{\text{app}} + C_{\text{obs}} + C_{\text{sec}}) + nf_{\text{data}}(r) + nf_{\text{interfaces}}(d, r) + \lambda n^\alpha + C_{\text{drift}}(n, r),$$

where $\alpha > 1$ captures coordination overhead, environment reservation conflicts, version drift, access management, and operational support. The cost can feel quasi-exponential in practice because dependencies multiply across environments.

The **Environment Representativeness Index (ERI)** can be approximated as:

$$ERI = w_D D + w_I I + w_C C + w_O O + w_S S + w_P P,$$

where D is data representativeness, I interface representativeness, C configuration fidelity, O observability fidelity, S security/access fidelity, and P performance fidelity. The weights are programme-specific and should be calibrated against historical defect escape rates: which representativeness gaps most often allow defects to escape staging undetected?

6.2 Non-Blocking Engineering Capacity

For programmes involving many actors — QA, BA, developers, security, integrators, product owners, customer representatives, support, data teams, sales engineers, and platform teams — the cost of a weak environment is not only risk. It is blocking. If ten actors depend on a single unstable qualification environment, the organization loses flow, credibility, and decision speed.

A blue/green strategy in qualification can therefore be justified even when it looks expensive from a server perspective. Blue remains the stable reference used for demonstrations, regression, customer validation, and security checks. Green receives the candidate version, configuration, migration, or interface change. The pattern buys continuity of work and reversibility before production.

A staging environment is not a server. It is a risk-compression asset and a team-flow preservation mechanism.

7. Deployment Patterns as Risk-Release Controls

The decision rule is not “use the most modern pattern.” The rule is:

Select the cheapest pattern that satisfies posterior constraints on residual risk, reversibility, team blocking, security exposure, and margin.

Table 2: Deployment and exposure patterns as risk-release controls.

Pattern	Primary Purpose	Best Suited For	Cost
Recreate / big bang	Simplicity; stop old, start new	Low-criticality, planned maintenance	Low
Rolling update	Progressive replacement, preserved availability	Stateless apps, Kubernetes, standard web services	Medium
Blue/green	Two equivalent environments, fast cutover and rollback	Critical releases, demos, qualification, rollback-sensitive systems	High
Canary	Small traffic fraction before generalization	Strong telemetry, low blast-radius segments	Medium-high

Pattern	Primary Purpose	Best Suited For	Cost
Ring deployment	Deploy by trust circles: internal, pilot, region, general	SaaS, B2B, multi-country rollout	Medium
Feature flags	Deploy without activating business exposure	Controlled activation, segmentation, logical rollback	Medium
A/B testing	Compare variants for value or behavior	UX, conversion, product discovery	Medium
A/A testing	Validate measurement and randomization pipeline	Experimentation platforms	Low-medium
Dark launch	Deploy hidden functionality to production	Integration testing without user exposure	Medium
Shadow traffic	Mirror real traffic to candidate without impacting users	ML, scoring, matching, migration, API replacement	High
Parallel run	Run old and new systems together and compare	Critical migration, identity, payment, public sector, registries	Very high
Expand/contract DB	Evolve schema with old and new code compatible	Zero-downtime database change	High
Strangler fig	Replace legacy incrementally	Long modernization programmes	High
Active/passive	Standby for failover	Disaster recovery	High
Active/active	Multiple active environments or regions	High availability, multi-region critical systems	Very high

8. SaaS versus On-Premise as a Governance Decision

The SaaS/on-premise choice is often framed as a technical architecture decision. In the proposed model, it is a governance decision involving at least nine dimensions:

1. **Time-to-value.** SaaS can accelerate delivery and bidding response.
2. **Data criticality.** Sensitive, sovereign, biometric, citizen, regulated, or classified data may impose constraints.
3. **Security capability.** Cloud/SaaS providers may offer stronger baseline security than a weak internal operation; conversely, they introduce supplier and concentration risks.
4. **Auditability.** Critical systems require evidence trails, logs, access records, configuration baselines, and contractual visibility.
5. **Exit strategy.** Lock-in risk must be priced, not ignored.
6. **Cost structure.** SaaS is often faster and lower-upfront; on-premise may be favorable for stable high-utilization workloads but requires CapEx, skills, operations, and lifecycle management.

7. **Sovereignty.** Jurisdiction, control plane, support access, and legal enforceability matter.
8. **Regulatory constraints.** GDPR, NIS2, DORA, EU AI Act, and sector-specific regulations may mandate specific placement choices as hard constraints regardless of economic analysis.
9. **Residual risk.** The selected option must be evaluated through posterior risk, not intuitive preference.

Table 3: Placement choices and dominant trade-offs.

Placement	Useful When	Watch-Outs
SaaS public	Speed, standard process, low customization, non-critical data	Vendor lock-in, data exposure, limited auditability, exit cost
SaaS sovereign / trusted cloud	Need SaaS speed with stronger jurisdictional control	Cost premium, limited catalog, contractual complexity
Public cloud IaaS/PaaS	Elasticity, cloud-native delivery, global scale	Cost drift, misconfiguration, shared responsibility gaps
Hybrid	Critical data local, elastic workloads remote	Integration complexity, duplicated skills, monitoring fragmentation
Private cloud / on-premise	Sovereignty, stable utilization, deep control	CapEx, patching, skills, observability, security maturity burden
Edge	Latency, offline operation, local capture, operational resilience	Fleet management, updates, physical security, data synchronization

Open source as a placement dimension. Open-weight AI models, open-source infrastructure, and self-hosted alternatives occupy a distinct position in this matrix. They can reduce vendor lock-in and token costs, but introduce supply chain risks (compromised dependencies, license compliance), maintenance burden, and security responsibility that must be explicitly modelled and resourced. Open source is a governance choice, not a free option.

9. AI Inference as Strategic Infrastructure

9.1 The AI-Token Trade-Off

The rise of AI APIs makes token consumption a new operational cost. Unlike traditional SaaS subscriptions, token costs scale with usage, output length, model quality, concurrency, and workflow design. Prompt engineering, retrieval-augmented generation, caching, summarization, batching, and model routing therefore become engineering economics concerns.

The AI inference decision has at least five options:

1. External proprietary API.
2. Managed cloud model platform.
3. Open-weight model hosted internally or in private cloud.
4. Enterprise internal model or fine-tuned model on dedicated compute.
5. Public, academic, non-profit, or sovereign compute infrastructure.

The choice can be modelled as:

$$\mathbf{a}_{\text{ai}}^* = f(C_{\text{token}}, C_{\text{gpu}}, \text{Latency}, \text{Quality}, \text{DataSensitivity}, \text{Sovereignty}, \\ \text{VendorDependence}, \text{Utilization}, \text{SecurityRisk}, \text{EthicalRisk}, \text{RegulatoryRisk}).$$

Model quality must be explicit. A cheaper model (open-weight or smaller) may have higher hallucination rates, lower accuracy on domain-specific tasks, or weaker robustness under adversarial inputs. These quality gaps translate into hidden costs: manual review workload, error correction, customer churn, or regulatory non-compliance. Quality metrics — task-specific accuracy, calibration, error rate under distribution shift — should be included in the adjusted margin calculation, not treated as a purely technical property.

Dynamic pricing. AI token pricing is highly volatile. Provider price changes, spot GPU compute fluctuations, and competitive pressure can alter the economics of a selected inference strategy within months of a bid. The adjusted margin should model token costs as stochastic distributions (informed by historical pricing volatility of the candidate providers) rather than point estimates, and include hedging provisions: multi-provider routing, output caching, and batch processing reserves.

9.2 Strategic Dependence

AI dependence is strategic because model access is not only a cost line. It includes provider policy, safety constraints, rate limits, pricing changes, geopolitical constraints, data retention terms, model deprecation, and explainability. A firm that builds critical engineering, QA, or commercial processes around a single external AI provider may gain speed but lose autonomy.

Ethical and reputational risk must be modelled alongside cost and sovereignty. For regulated sectors, an AI model that cannot explain its outputs (EU AI Act transparency requirements), that was trained on data with unclear provenance, or that produces outputs inconsistent with professional standards creates liability exposure that belongs in the adjusted margin. Scenario analysis — “what if our AI provider has a data breach or a model bias incident?” — should be part of governance review for any critical AI-dependent system.

Public AI compute initiatives such as EuroHPC AI Factories point to a possible counter-model: shared infrastructure supporting SMEs, startups, scientific users, and public-interest AI development. For sectors requiring sovereign inference, this represents a governance option between commercial API dependence and the full cost of internal GPU investment.

10. Stakeholder Governance Model

The confidence-constrained engineering economics framework is not owned by one function. It is a multi-actor decision. Table 4 maps stakeholder responsibilities.

Table 4: Actors in confidence-constrained governance.

Actor	Key Decision Interest	Required Evidence
Engineering lead	Feasible architecture, delivery risk, technical debt	Architecture options, complexity, dependencies, operational constraints
FinOps / finance	Budget, forecast, cost attribution, margin protection	Cost models, unit economics, spend forecast, allocation tags

Actor	Key Decision Interest	Required Evidence
Security / CISO	Exposure, supply chain, access, compliance	Threat models, SSDF evidence, SAST/SCA/DAST, audit logs, incident history
QA/BA / test governance	Residual risk, evidence coverage, MTP threshold strategy	Requirements-risk-test traceability, coverage, defects, logs, test data readiness
Platform / DevOps	Environment reliability, deployment pattern, observability	IaC plans, deployment telemetry, SLOs, rollback evidence, capacity metrics
Product / business	Customer value, feature timing, adoption, user risk	Business metrics, A/B results, customer commitments, user-impact analysis
Sales / presales	Bid competitiveness, contractual credibility, risk disclosure	Cost-risk assumptions, delivery scenario, security positioning, margin-risk analysis
Legal / compliance	Jurisdiction, data protection, liability, auditability	Contracts, data-flow maps, DPAs, regulatory requirements, sovereignty controls
Executive	Trade-off acceptance, investment, risk appetite	Pareto frontier, posterior risk, adjusted margin, decision options

Cognitive bias mitigation. The governance model should include structural protections against known decision biases. Optimism bias — teams systematically underestimating risk — is countered by reference class forecasting: use historical data from similar programmes as prior anchors. Sunk cost fallacy — doubling down on failing architectures — is countered by stage-gate reviews that treat prior investment as irretrievable and re-evaluate forward options on their own merits. Anchoring — over-reliance on initial cost estimates — is countered by requiring at least two independent cost models before a baseline is established. Pre-mortem sessions, in which teams are asked to assume the project has failed and explain why, are a low-cost structural mechanism for surfacing hidden risks before commitment.

11. Decision Artifacts for a Master Test Plan

In this framework, the MTP is not merely a documentation artifact. It becomes the operational contract that states how the organization will buy confidence.

A confidence-aligned MTP should include:

1. Risk register linked to business impact and system flows.
2. Explicit assumptions, hypotheses, evidence sources, and falsification criteria.
3. Environment representativeness level (ERI score) and known gaps.
4. Test data readiness and data-protection constraints.
5. Deployment pattern rationale and rollback evidence.
6. Security evidence aligned to SSDF and procurement constraints.
7. Engineering economics forecast: infra, staging, AI tokens, observability, and support costs.
8. Posterior decision thresholds for GO, Conditional GO, and NO-GO.
9. Residual risk acceptance owner and review date.

10. Margin-risk analysis, including expected residual loss and black swan buffer.

Automation and operationalization. An MTP that exists only as a document will function as shelfware. To be operationally effective it must be integrated into the delivery pipeline: automated checks that flag threshold breaches in CI/CD, live dashboards that track ERI and confidence scores by environment, and escalation paths that route GO/NO-GO decisions to the appropriate authority level when automated checks cannot resolve the gate. The MTP should describe not only what evidence is required but how that evidence is collected, where it lives, and what system triggers a gate review.

The MTP should not describe an ideal validation world. It should encode the economically acceptable threshold configuration under which a release or bid decision becomes defensible.

12. Illustrative Scenario

Consider a public-sector identity platform deliverable under a competitive bid. The platform involves citizen data, biometric interfaces, external registries, strict audit requirements, and a target bid price. The deployment options span: a SaaS workflow engine, a public cloud deployment, a sovereign cloud deployment, and an on-premise stack.

The naive financial model favors SaaS because it reduces implementation time. The security team raises data and jurisdiction concerns. The QA/BA team argues that a weak staging environment will not expose integration issues with external registries. Platform engineering warns that a fully representative pre-production environment will increase cost sharply. Presales wants a price that wins the bid.

Under the confidence-constrained model, each strategy vector $\mathbf{a}$ is evaluated against its posterior joint constraint:

- **SaaS public:** Low cost, high time-to-value, but sovereignty and audit constraints fail the regulatory hard filter for citizen biometric data. Strategy eliminated before probabilistic evaluation.
- **Sovereign cloud:** Higher cost, reduced jurisdictional risk, but staging representativeness for external registry interfaces remains low ($\text{ERI} \approx 0.6$). Residual integration risk exceeds τ_R . Conditional GO: requires interface simulators in qualification.
- **Public cloud + data localization:** Hybrid of elastic compute with data plane in national jurisdiction. Passes regulatory filter. Staging cost manageable with blue/green qualification environment. Canary deployment for non-critical services. Passes joint posterior constraint with $\gamma = 0.90$.
- **On-premise:** Highest control, sovereign by default, but the organization's security operations maturity is insufficient (insufficient patching cadence, no SOC). Posterior security constraint fails. Strategy requires prerequisite investment before it becomes viable.

The recommended strategy is the hybrid public cloud with data localization, blue/green qualification for external interface validation, canary release for non-critical services, and parallel run only for authoritative record migration — at a cost structure that preserves adjusted margin above τ_M with 0.91 posterior confidence.

This is the kind of mixed, evidence-bounded strategy that a confidence-constrained decision model makes explicit and auditable.

13. Research Agenda

The proposed framework opens several research directions. They are sequenced below by dependency: R1 and R2 provide the empirical foundations required by R3, R4, and R5.

R1: Risk-Adjusted Engineering Economics Metrics

Existing FinOps dashboards should be extended with residual-risk and confidence metrics. A useful metric is cost per unit of risk compression:

$$RCROI(\mathbf{a}) = \frac{\Delta \mathbb{E}[\text{Loss}_{\text{residual}}]}{\Delta C(\mathbf{a})}.$$

This measures residual loss reduction per additional unit of cost. Research task: establish which cost inputs most efficiently buy risk reduction across a sample of real programmes.

R2: Environment Representativeness Index Calibration

The ERI formulation (Section 6.1) requires empirical calibration: which representativeness dimensions most predict defect escape from staging to production? This requires programme-level data linking ERI scores to post-release incident rates. Research task: design the data collection protocol and calibrate weights across a multi-programme dataset.

R3: Deployment-Pattern Selection Under Uncertainty

R3 depends on calibrated risk distributions from R1 and R2. Research task: develop a pattern recommender that selects among rolling update, blue/green, canary, ring, shadow, or parallel run based on failure impact, telemetry maturity, reversibility, user segmentation, data risk, and budget.

R4: AI Inference Sovereignty and Quality Model

Formalize when an organization should move from external APIs to open-weight models, internal inference, or public compute. The model should include token cost distributions (not point estimates), latency, model quality metrics, data sensitivity, workload stability, GPU utilization, skill availability, and strategic dependence. R4 depends on R1 for the risk-adjusted cost metric.

R5: Bid Pricing with Residual-Risk Posterior

Extend commercial pricing models with posterior risk. A bid should not only include cost and target margin; it should include confidence that the adjusted margin remains acceptable after residual risk, black swan buffer, and execution uncertainty. R5 integrates R1–R4 into a bid governance protocol.

14. Limitations

This is a foundation article, not yet an empirical validation. Several limitations are acknowledged and define the research programme required to make confidence-constrained engineering economics operational.

Model abstraction. The mathematical model requires calibration on real projects. The posterior gate is analytically specified but practically dependent on prior elicitation quality and evidence standardization.

Evidence standardization. Estimating residual risk distributions is difficult when evidence is sparse, heterogeneous, or biased. Test evidence, incident data, threat models, and telemetry have different statistical properties and provenance. Aggregation methods require further development.

Dependency structure. The joint constraint formulation proposes a Gaussian copula as a tractable first approximation, but the true dependency structure between risk dimensions may be asymmetric, tail-dependent, or context-specific. Further empirical work is required.

Black swan events. The black swan buffer is a practical addition to the adjusted margin but does not resolve the fundamental problem: genuinely unprecedented events (SolarWinds-class supply chain attacks, Log4Shell-class zero-days, sudden regulatory regime changes) are not estimable from historical distributions. The buffer acknowledges this without solving it.

Human and organizational factors. The formal model is rational; organizations are not. Cognitive biases, power dynamics, organizational inertia, and incentive misalignment between finance, security, engineering, and sales can prevent adoption of the optimal strategy even when it is identified. The governance model (Section 10) provides structural mitigations, but implementation requires change management investment beyond the scope of this article.

Regulatory evolution. AI inference markets, sovereignty frameworks, and regulatory requirements (EU AI Act, NIS2, DORA) are evolving rapidly. The model's regulatory hard constraints require continuous update as the regulatory landscape changes.

15. Conclusion

Engineering economics began as a discipline of cost estimation under uncertainty. It should now evolve into a broader governance question: what does the cost of a technology strategy buy in terms of defensible confidence?

SaaS versus on-premise, staging depth, blue/green qualification, canary release, feature flags, security controls, sovereign cloud, AI token markets, open-weight self-hosting, internal GPU capacity, and public compute infrastructure are not separate decisions. They are coupled levers in a single governance problem: how to buy enough confidence without destroying margin, flow, security, or strategic autonomy.

The core thesis can be summarized in two statements:

FinOps measures what technology costs. Confidence-constrained engineering economics formalizes what that cost buys.

The new engineering trade-off is not cloud versus on-premise. It is the optimization of confidence under economic, security, sovereignty, delivery, and AI-compute constraints.

Future work should turn this foundation into a calibrated decision engine, using real programme histories, FinOps telemetry, test evidence, security findings, environment maturity scores, deployment telemetry, and Bayesian posterior updates to support GO, Conditional GO, NO-GO, redesign, hybridization, repatriation, or investment decisions. The research agenda in Section 13 is sequenced to build that calibration from the ground up.

References

- [1] FinOps Foundation. *FinOps Framework*. 2026. <https://www.finops.org/framework/>
- [2] FinOps Foundation. *FinOps Principles*. 2026. <https://www.finops.org/framework/principles/>
- [3] FinOps Foundation. *FinOps Phases: Inform, Optimize and Operate*. 2026. <https://www.finops.org/framework/phases/>
- [4] Saurabh Deochake. *ABACUS: A FinOps Service for Cloud Cost Optimization*. arXiv:2501.14753, 2024.

- [5] Murugiah Souppaya, Karen Scarfone, and Donna Dodson. *Secure Software Development Framework (SSDF) Version 1.1*. NIST SP 800-218, 2022.
- [6] ENISA. *Cloud Computing: Benefits, Risks and Recommendations for Information Security*. European Union Agency for Cybersecurity, 2009.
- [7] European Commission. *Cloud Sovereignty Framework*. 2025.
- [8] Ilari Raisanen. *Cloud Repatriation: A Strategic Move to Optimize IT Infrastructure and Costs*. 2024.
- [9] Kubernetes Documentation. *Deployments*. 2026.
- [10] Amazon Web Services. *Blue/Green Deployments on AWS*. 2026.
- [11] Martin Fowler. *Feature Toggles (aka Feature Flags)*. 2017.
- [12] Siavash Mirarab. *A Bayesian Framework for Software Regression Testing*. PhD thesis, University of Waterloo, 2008.
- [13] Michael Felderer and Rudolf Ramler. *A Bayesian Prediction Model for Risk-Based Test Selection*. Euromicro SEAA, 2015.
- [14] Chuan Fan and Yu. *BBN-based software project risk management*. Journal of Systems and Software, 73(2):193–203, 2004.
- [15] Hege Aalvik et al. *Establishing a Security Champion in Agile Software Teams*. SINTEF, 2023.
- [16] Muhammad Azeem Akbar et al. *Toward successful DevSecOps in software development organizations*. Information and Software Technology, 147:106894, 2022.
- [17] ISO/IEC/IEEE. *29119-3:2021 Software Testing — Part 3: Test documentation*. 2021.
- [18] Ege Erdil. *Inference Economics of Language Models*. arXiv:2506.04645, 2025.
- [19] Girish Sastry et al. *Computing Power and the Governance of Artificial Intelligence*. arXiv:2402.08797, 2024.
- [20] C. André et al. *Developments in Artificial Intelligence Markets*. OECD AI Papers No. 37, 2025.
- [21] Dirk Bergemann, Alessandro Bonatti, and Alex Smolin. *The Economics of Large Language Models*. arXiv:2502.07736, 2025.
- [22] Francisco Eiras et al. *Risks and Opportunities of Open-Source Generative AI*. arXiv:2405.08597, 2024.
- [23] EuroHPC Joint Undertaking. *AI Factories*. 2026.
- [24] Barry Boehm. *Software Engineering Economics*. Prentice-Hall, 1981.
- [25] Steve McConnell. *Software Estimation: Demystifying the Black Art*. Microsoft Press, 2006.
- [26] Douglas Hubbard. *How to Measure Anything: Finding the Value of Intangibles in Business*. Wiley, 2014.
- [27] Roger Cooke. *Experts in Uncertainty: Opinion and Subjective Probability in Science*. Oxford University Press, 1991.
- [28] Lawrence Phillips and Caitlin Wynn-Williams (eds.). *SHELF: The Sheffield Elicitation Framework*. University of Sheffield, 2011.